Natural Language Processing

NLP Assignment 3

English → Urdu Neural Machine Translation

RNN, BiRNN, LSTM, BiLSTM and Transformer, plus GloVe embeddings

Name	**Mohammad Ibrahim Salman**
Roll Number	**23L-0723**
Course	**Natural Language Processing**
Assignment	NLP Assignment 3

Result at a glance

Six translation models were trained on the same 24,524 English–Urdu sentence pairs with identical settings for 50 epochs each. The **Transformer** scores highest on every metric (**BLEU 39.70**, ROUGE-L 65.12, METEOR 58.80, chrF 56.99), even though it has the fewest parameters. Starting the RNN from pre-trained **GloVe** embeddings adds about 2.5 points on every metric and reaches the plain RNN's best validation loss in 22 epochs instead of 48.

Code: `23L-0723_NLP_Assignment-3.ipynb` (executed notebook) and `nmt_en_ur.py` (the same code as a script). Hardware: Kaggle notebook on an NVIDIA Tesla T4 GPU. Full training log: `results/training_log.txt`.

Contents

1 Setup **2**
1.1 Dataset and preprocessing (Part 1) . . . 2
1.2 Shared training configuration (identical for all models) . . . 2
1.3 Models (Part 2) . . . 3

2 Results on the held-out test set (Part 3) **4**
2.1 The metrics . . . 4
2.2 Final scores . . . 5
2.3 Inference examples . . . 6

3 Part 4: random vs. pre-trained GloVe embeddings (RNN) **8**

4 Conclusion: which model performed best and why? **9**

5 Reproducing **10**

A All fixed test sentences **10**

How to read the colours in this document

Teal	data and outside knowledge: the corpus, the GloVe vectors, the "RNN + GloVe" model
Navy	model machinery and the LSTM family
Slate	the baseline: the plain RNN family
Vermilion	the thing to notice: the best model (Transformer), attention, key findings
Grey	structure: arrows, guides, labels

In plots, *dashed* lines or hollow markers mean a *bidirectional* encoder, so the figures still read in greyscale.

1 Setup

The task is **machine translation**: a model reads an English sentence and writes the Urdu translation. Every model here is an **encoder–decoder**. The *encoder* reads the English sentence and turns it into numbers. The *decoder* uses those numbers to produce the Urdu sentence one word at a time.

1.1 Dataset and preprocessing (Part 1)

Dataset: Parallel corpus for English & Urdu (Kaggle). It is a **parallel corpus**: two text files where line i in one file is the translation of line i in the other. After removing empty lines there are **24,524 sentence pairs**. Most are short conversational sentences of 3–6 words. Figure 1 shows the preprocessing steps.

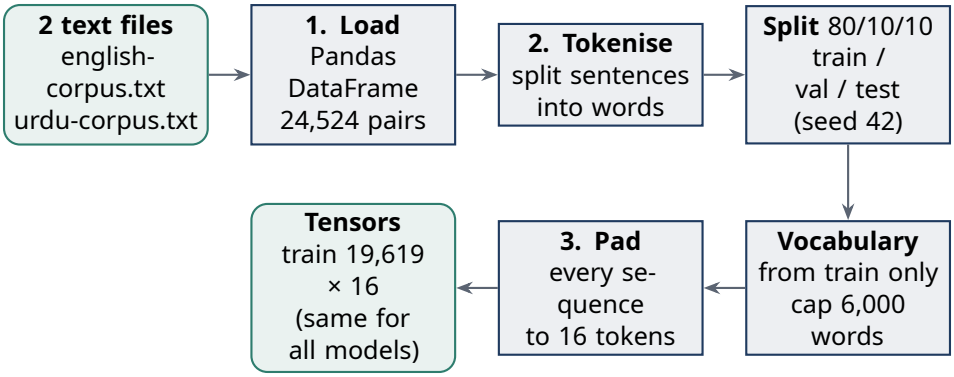


Figure 1. Preprocessing turns two raw text files into fixed-size number grids that every model shares. Rounded teal boxes are data; square navy boxes are operations. Steps 1–3 are the three Part 1 tasks.

1. **Load.** Both text files were loaded into a Pandas *DataFrame* (a table with an `english` and an `urdu` column). Empty lines were dropped.
2. **Tokenise.** A **token** is one unit the model sees, here a word or a punctuation mark. *Tokenising* means splitting a sentence into tokens.
   - **English:** lower-cased word tokens, with punctuation split off.
   - **Urdu:** Urdu punctuation (. ؟ ،) is split off, then the text is split on whitespace.
3. **Split** with a fixed seed into 80% train (19,619 pairs), 10% validation (2,452) and 10% test (2,453). The *validation* set is used to pick the best checkpoint. The *test* set is only used for the final scores.
4. **Build vocabularies** from the training split only. A **vocabulary** is the list of known tokens; each gets an ID number. Both are capped at **6,000 tokens** and include the special tokens `<pad>` `<sos>` `<eos>` `<unk>` (padding, start of sentence, end of sentence, unknown word). The English vocabulary has 5,135 entries and the Urdu one has 5,134.
5. **Pad** every source and target sequence to **16 tokens**. **Padding** fills short sequences with `<pad>` so they all have the same length and can be stacked into one grid. Target sequences are wrapped in `<sos>` … `<eos>`. This covers more than 99% of sentences without truncation (cutting words off).

The vocabulary and padding are shared by every model, as the assignment requires.

1.2 Shared training configuration (identical for all models)

Table 1 lists the settings. Every model uses exactly these values, so differences in the results come from the architecture alone.

Table 1. Shared training configuration. Every model uses these exact values.

Setting	Value	What it means
Embedding dim	200 (matches GloVe 6B-200d)	length of the vector that represents each word
Hidden size	256	size of the model's internal memory vector
Layers	2 encoder + 2 decoder	how many processing layers are stacked
Dropout	0.3	randomly switch off 30% of units while training
Optimiser	Adam, lr = 5e-4	the rule for updating weights, and its step size
Batch size	64	sentence pairs processed per update
Epochs	**50**	full passes over the training data
Loss	cross-entropy, label smoothing 0.1, padding ignored	how wrong a prediction is
Gradient clipping	1.0	caps the size of each update
Decoding	greedy, max 16 tokens	how translations are produced at test time
Checkpoint	lowest validation loss	which saved weights are evaluated
Seed	42	fixes randomness so runs repeat exactly

Key terms used in the configuration

- **Embedding**: a learned vector of numbers that stands for a word. Words used in similar ways end up with similar vectors.
- **Epoch**: one full pass through all 19,619 training pairs.
- **Dropout**: during training, randomly ignore some units so the model cannot rely on any single one. This reduces *overfitting*.
- **Overfitting**: the model memorises the training sentences instead of learning rules that work on new sentences.
- **Cross-entropy loss**: a number that is low when the model gives high probability to the correct next word. Lower is better.
- **Label smoothing (0.1)**: the training target gives 90% to the correct word and spreads 10% over the others, so the model does not become over-confident.
- **Gradient clipping**: if an update would be larger than a limit (1.0), it is scaled down. This stops training from blowing up.
- **Teacher forcing**: during training the decoder is fed the *correct* previous Urdu word, not its own guess.
- **Greedy decoding**: at test time, the decoder picks the single most likely word at each step until it outputs `<eos>`.

1.3 Models (Part 2)

Figure 2 contrasts the two kinds of architecture. The key difference is how much of the English sentence the decoder can see.

- **RNN / LSTM:** a sequence-to-sequence (*seq2seq*) encoder–decoder in the style of Cho et al. (2014).
  - An **RNN** (recurrent neural network) reads words one at a time and updates a memory vector after each word.
  - An **LSTM** (long short-term memory) is an RNN with *gates*: small learned switches that decide what to keep, forget and output. This lets it remember information over

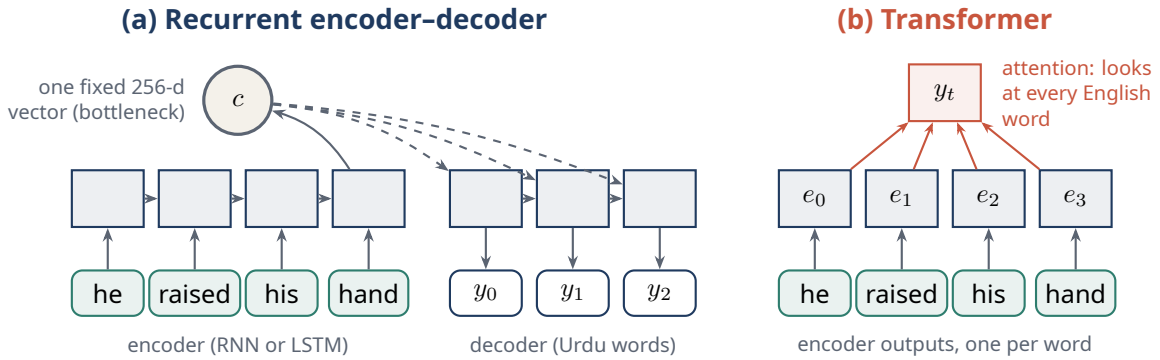


Figure 2. (a) The recurrent models squeeze the whole English sentence into *one* vector c, which every decoder step must reuse. (b) The Transformer's decoder uses attention to look directly at every English word at every step, so nothing has to pass through a bottleneck. This is a schematic, not real model values.

longer spans.

- The encoder is a 2-layer Elman RNN or LSTM. Sequences are *packed* so padding is ignored.
- The final state initialises the decoder.
- The top-layer context vector is also fed to the decoder at every step and to the output layer.
- These models have no attention.

- **BiRNN / BiLSTM:** the same as above, but with a **bidirectional** encoder. It reads the sentence both left-to-right and right-to-left. The forward and backward final states of each layer are concatenated (joined end to end) and projected with Linear + tanh to the decoder's hidden size.
- **Transformer:** `nn.Transformer` with 2 encoder and 2 decoder blocks, 8 attention heads, $d_{\text{model}} = 200$ and a feed-forward size of 512.
  - **Attention** lets each output word compute a weighted mix of all encoder outputs, focusing on the most relevant English words.
  - **Multi-head attention** runs 8 of these in parallel, so different heads can focus on different things (for example word order vs. word meaning).
  - It uses sinusoidal *positional encodings* (fixed sine/cosine patterns that tell the model each word's position), padding masks and a *causal* decoder mask (the decoder cannot peek at future words).
- **RNN + GloVe:** the same RNN, but the English embedding layer starts from `glove.6B.200d` vectors and is then fine-tuned (allowed to keep changing during training). GloVe covers **98.2%** of the English vocabulary (5,037 of 5,131 words).

2 Results on the held-out test set (Part 3)

All models were evaluated on the same **2,453 test sentences**, which no model saw during training.

2.1 The metrics

All scores are on a 0–100 scale; higher is better.

Evaluation metrics

- **BLEU-4**: counts how many 1-, 2-, 3- and 4-word sequences (*n-grams*) in the model's output also appear in the reference translation. It is computed over the whole test set (corpus-level). BLEU-1 counts single words only.

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} \tfrac{1}{4} \log p_n\right)$$

  In words: p_n is the fraction of the output's n-grams that match the reference, the four are averaged (geometrically), and BP is a *brevity penalty* that lowers the score if outputs are too short.

- **ROUGE-1 / ROUGE-2 / ROUGE-L**: F1 overlap between output and reference, using single words, word pairs, and the *longest common subsequence* (the longest run of words appearing in the same order in both), averaged over sentences.
- **METEOR**: matches words between output and reference, balances precision and recall, and penalises outputs where matched words are scattered (fragmentation). Averaged over sentences.
- **chrF**: an F-score over character sequences (1–6 characters). It gives partial credit for nearly-right word forms.

Notes on the metrics

- **ROUGE** is computed in the notebook on Urdu word tokens. The common `rouge_score` package's default tokenizer drops non-Latin characters, so it would score every Urdu sentence as 0.
- **METEOR** uses exact and stem matches only, because there is no Urdu WordNet (WordNet is an English synonym dictionary).
- **chrF** is a character-level check that does not depend on how words are tokenised.

2.2 Final scores

Table 2 gives every score; Figure 3 shows the four main metrics side by side.

Table 3 shows that the **LSTM** has the lowest validation loss, yet the Transformer translates best. Loss measures how confidently the model predicts each next word *when given the correct previous words*; the test scores measure the full translations the model writes on its own. The two do not have to agree exactly.

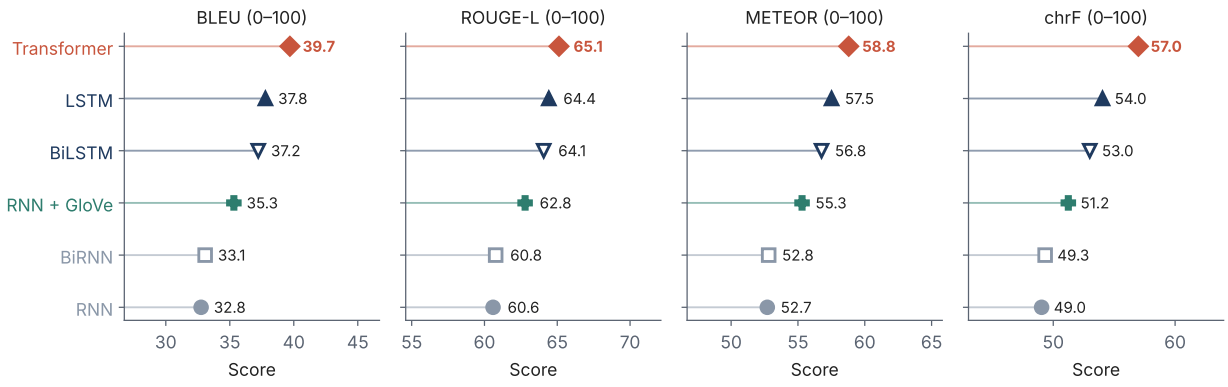


Figure 3. The Transformer (vermilion diamond) leads on all four metrics. LSTMs (navy) beat plain RNNs (slate) by about 5 points, and GloVe (teal) lifts the RNN about halfway to the LSTMs. Hollow markers are bidirectional encoders.

Table 2. Test-set scores (0–100, higher is better). The Transformer is best on every metric.

Model	BLEU-4	BLEU-1	ROUGE-1	ROUGE-2	ROUGE-L	METEOR	chrF
RNN	32.76	59.74	60.93	38.82	60.59	52.71	49.04
BiRNN	33.07	59.73	61.14	39.44	60.77	52.81	49.33
LSTM	37.78	64.40	64.92	43.80	64.42	57.52	54.04
BiLSTM	37.22	63.95	64.45	42.99	64.07	56.77	53.00
Transformer	**39.70**	**65.08**	**65.75**	**45.54**	**65.12**	**58.80**	**56.99**
RNN + GloVe	35.32	62.28	63.19	41.16	62.79	55.31	51.24

Table 3. Model size, best validation loss and training cost (50 epochs on a Tesla T4).

Model	Best val loss	Best epoch	Params	Train time (50 ep)	s / epoch
RNN	2.885	48	5.25 M	162 s	3.2
BiRNN	2.882	49	5.76 M	177 s	3.5
LSTM	**2.747**	48	6.94 M	212 s	4.2
BiLSTM	2.790	29	8.72 M	246 s	4.9
Transformer	2.824	50	4.88 M	289 s	5.8
RNN + GloVe	2.785	48	5.25 M	161 s	3.2

Figure 4 shows the validation loss for each model across all 50 epochs.

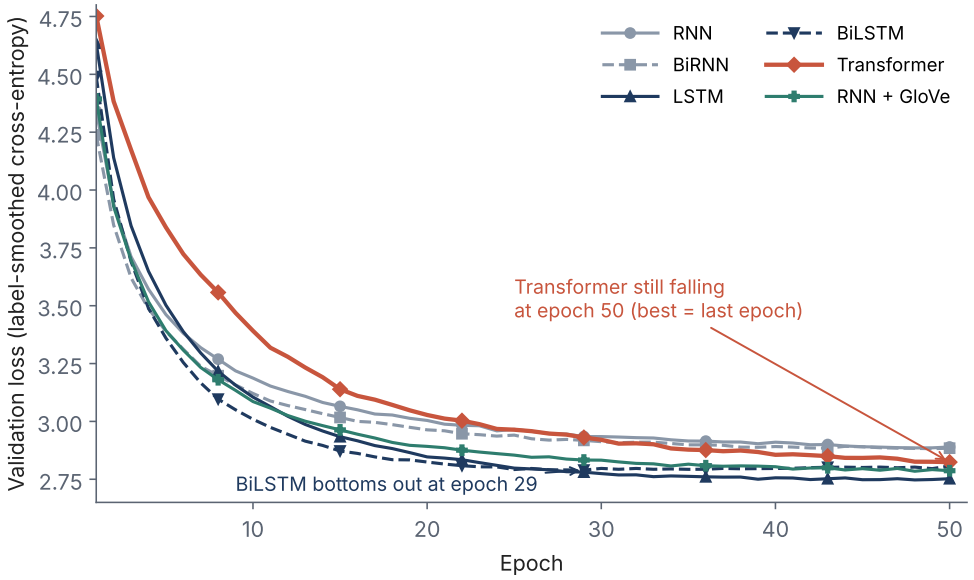


Figure 4. Validation loss per epoch. The recurrent models flatten out well before epoch 50, but the Transformer is still improving at the end. Its best checkpoint is the last epoch, so it had not finished learning.

2.3 Inference examples

Table 4 shows five of the 15 fixed test sentences. The other ten are in Appendix A. ✓ marks an output identical to the reference, ignoring spacing and final punctuation. Urdu is written right to left.

Table 4. Five fixed test sentences translated by every model.

Model	Urdu output	= ref
EN: *he is an italian* Reference: وہ ایک اطالوی ہے		
RNN	وہ ایک excon ہے	
BiRNN	وہ ایک انگریز ہے	
LSTM	وہ ایک ماہر ہے	
BiLSTM	وہ ایک ماہر ہے	
Transformer	وہ ایک اطالوی ہے	✓
RNN + GloVe	وہ ایک فرانسیسی ہے	
EN: *zain is seldom home* Reference: زین شاذ و نادر ہی گھر ہوتا ہے۔		
RNN	زین گھر گھر ہے	
BiRNN	زین گھر سے گھر ہے ۔	
LSTM	زین گھر سے گھر ہے ۔	
BiLSTM	زین گھر سے گھر ہے ۔	
Transformer	زین شاذ و نادر ہی گھر ہے ۔	
RNN + GloVe	زین گھر سے گھر ہے ۔	
EN: *i need a new bicycle* Reference: مجھے ایک نئی سائیکل چاہیے		
RNN	مجھے ایک نئی سائیکل چاہیے	✓
BiRNN	مجھے ایک نئی سائیکل چاہیے	✓
LSTM	میں ایک نئی سائیکل کی ضرورت ہے	
BiLSTM	میں ایک نئی الماری کی ضرورت ہے	
Transformer	مجھے ایک نئی سائیکل چاہیے	✓
RNN + GloVe	مجھے ایک نئی سائیکل چاہیے	✓
EN: *i can sleep anywhere* Reference: میں کہیں بھی سو سکتا ہوں		
RNN	میں کہیں بھی جا سکتا ہوں ۔	
BiRNN	میں کہیں کہیں ہوں	
LSTM	میں بھی سو سکتا ہوں	
BiLSTM	میں کہیں بھی جا سکتا ہوں	
Transformer	میں کہیں بھی سو سکتا ہوں	✓
RNN + GloVe	میں کہیں بھی سو سکتا ہوں	✓
EN: *were leaving* Reference: جا رہے تھے		
RNN	مورخ تھے	
BiRNN	جا رہے تھے	✓
LSTM	جا رہے تھے	✓
BiLSTM	جا رہے تھے	✓
Transformer	جا رہے تھے	✓
RNN + GloVe	چھپے تھے	

zain is seldom home: the Transformer is the only model that produces شاذ و نادر ("seldom"). Its output differs from the reference only by the word ہوتا, so it does not get a ✓.

Table 5 shows custom sentences that are not in the dataset (also saved as `results/custom_translations.csv`).

Table 5. Custom sentences that are not in the dataset. Identical outputs are grouped; the right column lists which models produced each one.

English	Urdu output	Produced by
i am happy	میں خوش ہوں	all 6 models
where is my book	میری کتاب کہاں ہے ؟	RNN, LSTM, BiLSTM, RNN + GloVe
	میری کتاب کہاں ہے	BiRNN, Transformer

English	Urdu output	Produced by
he is a good boy	وہ ایک اچھا لڑکا ہے وہ اچھا لڑکا ہے ۔ وہ ایک اچھا لڑکا ہے ۔	RNN, BiRNN, Transformer, RNN + GloVe LSTM BiLSTM
what is your name	ہائے آپ کا نام کیا ہے	RNN, BiRNN, LSTM, BiLSTM, RNN + GloVe Transformer
we are going home	ہم گھر جا رہے ہیں	all 6 models

What the custom sentences show

All recurrent models translate “what is your name” as ہائے (“hi”). They have learned a greeting-like mapping from the fixed context vector. The **Transformer** attends to the individual words and gets the translation right: آپ کا نام کیا ہے.

The plots are also saved as image files: `results/loss_curves.png` (training and validation loss for all models), `results/scores.png` (test scores by model) and `results/glove_vs_random.png` (RNN with random vs. GloVe embeddings).

3 Part 4: random vs. pre-trained GloVe embeddings (RNN)

GloVe (Global Vectors) is a set of word embeddings pre-trained on 6 billion words of English text. Instead of starting the RNN’s English embedding layer from random numbers, we start it from these vectors, which already place related words close together. The RNN was trained twice with everything else identical. Table 6 compares the two runs.

Table 6. RNN with random vs. GloVe English embeddings. GloVe is better on every metric at the same cost per epoch.

	RNN (random emb.)	RNN + GloVe	Δ
BLEU-4	32.76	**35.32**	**+2.56**
ROUGE-L	60.59	**62.79**	+2.20
METEOR	52.71	**55.31**	+2.60
chrF	49.04	**51.24**	+2.20
Best validation loss	2.885 (epoch 48)	**2.785** (epoch 48)	−0.10
Epochs to reach the random model’s best val loss (2.885)	48	**22**	**2.2× faster**
Wall-clock training time (50 epochs)	162 s	161 s	≈ same
Time per epoch	3.23 s	3.22 s	≈ same

Figure 5 shows the loss curves for both runs.

Discussion

- **Performance.** GloVe initialisation improves every metric by 2–2.6 points (BLEU +2.6, METEOR +2.6). That is almost 60% of the gap between the plain RNN and the LSTM, with no extra parameters. The corpus is small (about 20k sentences), so many English words appear only a few times. With random initialisation the model must learn their meaning from those few examples. GloVe vectors are trained on 6B tokens and already place related words close together (*italian* / *french* / *english*, *happy* / *glad*). The encoder therefore gets meaningful input from the first epoch and generalises better to rare words.

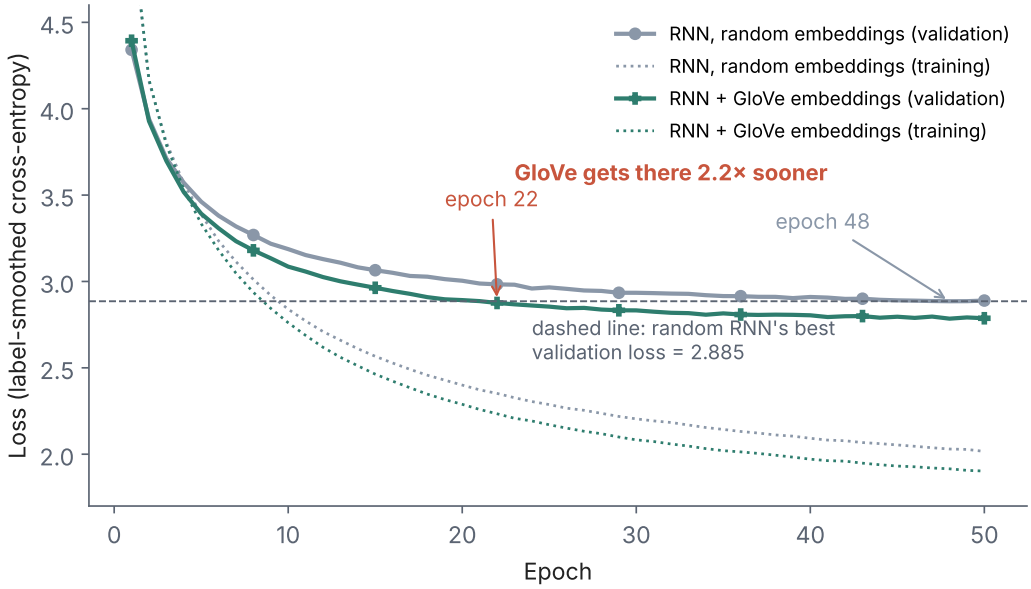


Figure 5. With GloVe the RNN's validation loss falls faster and ends lower. It crosses the random model's *best* value (dashed line) at epoch 22; the random model only gets there at epoch 48.

- **Training time.** The cost per epoch is identical (3.2 s) because the architecture and parameter count do not change. Reading the GloVe file and building the embedding matrix is a one-off cost of a few seconds. Convergence is much faster: the GloVe model reaches the random model's *best* validation loss at epoch 22 instead of epoch 48. It is also ahead from the first few epochs (epoch 10 val loss: 3.09 vs 3.19). To reach a given quality you need roughly half the training time.
- **Limits.** The gains come only from the encoder, because GloVe does not cover Urdu. They are bounded by the RNN's fixed-size context vector. Some outputs show GloVe's semantic neighbours being confused, e.g. *italian* → فرانسیسی (French). Similar words have similar vectors, and the small decoder cannot always tell them apart.

Part 4 answer

Pre-trained GloVe embeddings make the RNN **better** (about +2.5 on every metric) and **converge faster** (the same validation loss in 22 epochs instead of 48), at **no extra cost per epoch**.

4 Conclusion: which model performed best and why?

Answer

The Transformer is the best model. It has the highest score on every metric: BLEU 39.7, ROUGE-L 65.1, METEOR 58.8 and chrF 57.0. It also has the *fewest* parameters (4.9 M). The ranking is

Transformer > LSTM ≈ BiLSTM > RNN + GloVe > BiRNN ≈ RNN

Why:

1. **Attention replaces the bottleneck.** Every recurrent model squeezes the source sentence into one 256-d vector (Figure 2a). The Transformer's multi-head cross-attention lets each Urdu output word look directly at the relevant English words. This shows on rare and content words that have to be copied over precisely, e.g. *italian* → اطالوی, *seldom* → شاذ و نادر,

your name → آپ کا نام, which the recurrent models replace with frequent words. It also shows in ROUGE-2 and chrF, which reward correct word order and correct word forms; there the Transformer leads by 1.7 and 3 points.

2. **Gating matters for RNNs.** LSTMs beat vanilla RNNs by about 5 BLEU. The gates protect the *gradient* (the learning signal) and the memory, so the final encoder state keeps more of the sentence. The vanilla RNN's state is dominated by the last few words.
3. **Bidirectionality helps little here.** BiRNN gains 0.3 BLEU over RNN, and BiLSTM is slightly *worse* than LSTM (-0.6). Without attention, the decoder only sees the final states, and the sentences are short (about 5 tokens), so the forward state already sees most of the sentence. BiLSTM also has 26% more parameters. It overfits sooner: its best validation loss came at epoch 29 and then rose, while the training loss was the lowest of all models.
4. **The Transformer had not finished learning.** Its validation loss was still falling at epoch 50, and its best checkpoint is the last one (Figure 4). The other models had plateaued. With more epochs or a learning-rate warm-up schedule, the gap would probably grow. Its higher time per epoch (5.8 s vs 3–5 s) comes from the self-attention and cross-attention layers, which do more work per token than a 2-layer RNN on such short sequences. It is still under 5 minutes in total on a T4.

Overall, for this small, short-sentence English→Urdu corpus, **attention matters more than recurrence type, and recurrence type matters more than bidirectionality.** Pre-trained embeddings are a cheap way to improve a weak model when data is scarce.

5 Reproducing

On Kaggle (GPU). Push the kernel in the `kaggle/` folder. This trains everything from scratch, about 25 minutes on a T4.

```
kaggle kernels push -p kaggle --accelerator NvidiaTeslaT4
```

Locally. Put `english-corpus.txt`, `urdu-corpus.txt` and `glove.6B.200d.txt` under `DATA_DIR`.

```
DATA_DIR=data python nmt_en_ur.py
```

To evaluate the trained weights in `results/` without retraining, set `PRETRAINED_DIR=results`. This is how `23L-0723_NLP_Assignment-3.ipynb` was executed.

A All fixed test sentences

The remaining ten of the 15 fixed test sentences (all 15 are in `results/sample_translations.csv`). ✓ marks an output identical to the reference, ignoring spacing and final punctuation.

Table 7. The other ten fixed test sentences.

Model	Urdu output	= ref
EN: *whose land is this*	Reference: جس کی زمین یہ ہے	
RNN	یہ کس کا ہے	
BiRNN	یہ کس کا گٹار ہے	
LSTM	یہ کس کی پنسل ہے	
BiLSTM	یہ کس کی قمیض ہے ؟	
Transformer	یہ کس کا مالک ہے	
RNN + GloVe	یہ کس کا گٹار ہے	

Model	Urdu output	= ref
EN: *what is it worth* Reference: اس کی کیا قیمت ہے		
RNN	یہ کیا ہے	
BiRNN	یہ کیا ہے	
LSTM	یہ کیا کام ہے	
BiLSTM	یہ کیا ہوا ہے	
Transformer	یہ کیا قابل ہے	
RNN + GloVe	یہ کس چیز ہے	
EN: *does that satisfy you* Reference: کیا یہ آپ کو مطمئن کرتا ہے؟		
RNN	کیا یہ آپ کو جانتی ہے ؟	
BiRNN	کیا یہ آپ کو تکلیف دیتا ہے	
LSTM	کیا اس سے آپ خوفزدہ ہوتے ہیں	
BiLSTM	کیا یہ آپ سے محبت کرتا ہے	
Transformer	کیا آپ اس سے لطف اندوز ہوتا ہے	
RNN + GloVe	کیا یہ آپ کی بات کرتا ہے ؟	
EN: *how was your night* Reference: تمہاری رات کیسی گزری		
RNN	آپ کا ویک کیسا رہا ؟	
BiRNN	آپ کا ویک کیسا تھا ؟	
LSTM	آپ کا ویک کیسا تھا	
BiLSTM	آپ کا ویک کیسا رہا تھا	
Transformer	آپ کا رات کیسا تھا	
RNN + GloVe	آپ کی رات کیسا تھی ؟	
EN: *zain saw everything* Reference: زین نے سب کچھ دیکھا		
RNN	زین نے سب کچھ دیکھا	✓
BiRNN	زین نے سب کچھ دیکھا	✓
LSTM	زین نے سب کچھ دیکھا	✓
BiLSTM	زین نے سب کچھ دیکھا	✓
Transformer	زین نے سب کچھ دیکھا	✓
RNN + GloVe	زین نے سب کو دیکھا	
EN: *dont spill the beans* Reference: پھلیاں نہ پھیلائیں		
RNN	اس کی مت مت کرو	
BiRNN	دروازہ نہ دیں	
LSTM	کتے کو مت کھولو	
BiLSTM	دروازہ نہ کرو	
Transformer	احتیاط مت توڑو	
RNN + GloVe	باورچی نہ پیو	
EN: *he raised his hand* Reference: اس نے اپنا ہاتھ اٹھایا		
RNN	اس نے اپنا ہاتھ اٹھایا	✓
BiRNN	اس نے اپنا ہاتھ اٹھایا	✓
LSTM	اس نے اپنا ہاتھ اٹھایا	✓
BiLSTM	اس نے اپنا ہاتھ اٹھایا	✓
Transformer	اس نے اپنا ہاتھ اٹھایا	✓
RNN + GloVe	اس نے اپنا ہاتھ اٹھایا	✓
EN: *you broke my heart* Reference: تم نے میرا دل توڑا		
RNN	آپ نے میرا دل توڑ دیا ۔	
BiRNN	تم نے میرا دل توڑ دیا ۔	
LSTM	تم نے میرا دل توڑ دیا ۔	
BiLSTM	تم نے میرا دل توڑ دیا ۔	
Transformer	تم نے میرا دل توڑ دیا ۔	
RNN + GloVe	تم نے میرا دل توڑ دیا ۔	

Model	Urdu output	= ref
EN: *the house was decorated* Reference: گھر سجایا گیا تھا		
RNN	گھر بند تھا	
BiRNN	گھر بیمار تھا	
LSTM	گھر پریتوادت تھا	
BiLSTM	گھر خالی تھا	
Transformer	گھر پریتوادت تھا	
RNN + GloVe	گھر گھر تھا	
EN: *i am at your service* Reference: میں آپ کی خدمت میں حاضر ہوں۔		
RNN	میں آپ کے قریب ہوں ۔	
BiRNN	میں آپ کے بیٹے کے لیے ہوں ۔	
LSTM	میں آپ کے اختیار میں ہوں	
BiLSTM	میں آپ کی عمر میں ہوں	
Transformer	میں آپ کے بالکل پاگل ہوں	
RNN + GloVe	میں آپ کے ساتھ ہوں ۔	